

Project organization

Introduction

Since Godot has no restrictions on project structure or filesystem usage, organizing files when learning the engine can seem challenging. This tutorial suggests a workflow which should be a good starting point. We will also cover using version control with Godot.

Organization

Godot is scene-based in nature, and uses the filesystem as-is, without metadata or an asset database.

Unlike other engines, many resources are contained within the scene itself, so the amount of files in the filesystem is considerably lower.

Considering that, the most common approach is to group assets as close to scenes as possible; when a project grows, it makes it more maintainable.

As an example, one can usually place into a single folder their basic assets, such as sprite images, 3D model meshes, materials, and music, etc. They can then use a separate folder to store built levels that use them.

```
/project.godot
/docs/.gdignore # See "Ignoring specific folders" below
/docs/learning.html
/models/town/house/house.dae
/models/town/house/window.png
/models/town/house/door.png
/characters/player/cubio.dae
/characters/player/cubio.png
/characters/enemies/goblin/goblin.dae
/characters/enemies/goblin/goblin.png
/characters/npcs/suzanne/suzanne.dae
/characters/npcs/suzanne/suzanne.png
/levels/riverdale/riverdale.scn
```

Style guide

For consistency across projects, we recommend following these guidelines:

- Use **snake_case** for folder and file names (with the exception of C# scripts). This sidesteps case sensitivity issues that can crop up after exporting a project on Windows. C# scripts are an exception to this rule, as the convention is to name them after the class name which should be in PascalCase.
- Use **PascalCase** for node names, as this matches built-in node casing.
- In general, keep third-party resources in a top-level **addons/** folder, even if they aren't editor plugins. This makes it easier to track which files are third-party. There are some exceptions to this rule; for instance, if you use third-party game assets for a character, it makes more sense to include them within the same folder as the character scenes and scripts.

Importing

Godot versions prior to 3.0 did the import process from files outside the project. While this can be useful in large projects, it resulted in an organization hassle for most developers.

Because of this, assets are now transparently imported from within the project folder. See [Import process](#) for more details on how importing works in Godot.

Ignoring specific folders

To prevent Godot from importing files contained in a specific folder, create an empty file called **.gdignore** in the folder (the leading **.** is required). This can be useful to speed up the initial project importing.

Note

To create a file whose name starts with a dot on Windows, place a dot at both the beginning and end of the filename (**.gdignore.**). Windows will automatically remove the trailing dot when you confirm the name.

Alternatively, you can use a text editor such as Notepad++ or use the following command in a command prompt: **type nul > .gdignore**

Once the folder is ignored, resources in that folder can't be loaded anymore using the **load()** and **preload()** methods. Ignoring a folder will also automatically hide it from the FileSystem dock, which can be useful to reduce clutter.

Note that the **.gdignore** file's contents are ignored, which is why the file should be empty. It does not support patterns like **.gitignore** files do.

Case sensitivity

Windows and recent macOS versions use case-insensitive filesystems by default, whereas Linux distributions use a case-sensitive filesystem by default. This can cause issues after exporting a project, since Godot's PCK virtual filesystem is case-sensitive. To avoid this, it's recommended to stick to `snake_case` naming for all files in the project (and lowercase characters in general).

Note

You can break this rule when style guides say otherwise (such as the C# style guide). Still, be consistent to avoid mistakes.

On Windows 10, to further avoid mistakes related to case sensitivity, you can also make the project folder case-sensitive. After enabling the Windows Subsystem for Linux feature, run the following command in a PowerShell window:

To enable case-sensitivity:

```
fsutil file setcasesensitiveinfo <path to project folder> enable
```

To disable case-sensitivity:

```
fsutil file setcasesensitiveinfo <path to project folder> disable
```

If you haven't enabled the Windows Subsystem for Linux, you can enter the following line in a PowerShell window *running as Administrator* then reboot when asked:

```
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Windows-Subsystem-Linux
```